

Erstellung einer Applikation zur Generierung und digitalen
Signierung elektronischer Dokumente (PDF)

Creating an application to generate and digitally sign electronic do-
cuments (PDF)

Philipp Schmitz

Bachelor-Abschlussarbeit

Betreuer: Prof. Dr. Konstantin Knorr

Ort, DD.MM.YYYY

Vorwort

Ein Vorwort ist nicht unbedingt nötig. Falls Sie ein Vorwort schreiben, so ist dies der Platz, um z.B. die Firma vorzustellen, in der diese Arbeit entstanden ist, oder um den Personen zu danken, die in irgendeiner Form positiv zur Entstehung dieser Arbeit beigetragen haben.

Auf keinen Fall sollten Sie im Vorwort die Aufgabenstellung näher erläutern oder vertieft auf technische Sachverhalte eingehen.

Kurzfassung

In der Kurzfassung soll in kurzer und prägnanter Weise der wesentliche Inhalt der Arbeit beschrieben werden. Dazu zählen vor allem eine kurze Aufgabenbeschreibung, der Lösungsansatz sowie die wesentlichen Ergebnisse der Arbeit. Ein häufiger Fehler für die Kurzfassung ist, dass lediglich die Aufgabenbeschreibung (d.h. das Problem) in Kurzform vorgelegt wird. Die Kurzfassung soll aber die gesamte Arbeit widerspiegeln. Deshalb sind vor allem die erzielten Ergebnisse darzustellen. Die Kurzfassung soll etwa eine halbe bis ganze DIN-A4-Seite umfassen.

Hinweis: Schreiben Sie die Kurzfassung am Ende der Arbeit, denn eventuell ist Ihnen beim Schreiben erst vollends klar geworden, was das Wesentliche der Arbeit ist bzw. welche Schwerpunkte Sie bei der Arbeit gesetzt haben. Andernfalls laufen Sie Gefahr, dass die Kurzfassung nicht zum Rest der Arbeit passt.

Abstract

The same in English.

Inhaltsverzeichnis

1	Einleitung und Problemstellung	1
2	Grundlagen	2
2.1	Mathematische Grundlagen	2
2.1.1	Dualsystem (Binärsystem)	2
2.1.2	Hexadezimalsystem	2
2.1.3	Division mit Rest	3
2.1.4	Wiederholtes Quadrieren	3
2.1.5	Primzahlen und Teilbarkeit	3
2.1.6	Größter gemeinsamer Teiler	4
2.1.7	Probleme aus der Zahlentheorie	4
2.2	Grundwerte der Informationssicherheit	5
2.3	Hashfunktionen	6
2.4	Digitale Signaturen	7
2.4.1	Qualifizierte elektronische Signatur	7
2.4.2	Überblick	8
2.4.3	RSA (Rivest-Shamir-Adleman)	8
2.4.4	DSA (Digital Signature Algorithm)	10
2.5	PKI (Public-Key Infrastructure)	11
2.5.1	X.509 Version 3 Zertifikat	12
2.5.2	Gültigkeitsmodelle	12
2.6	PDF (Portable Document Format)	13
2.6.1	Objekte	13
2.6.2	Datei-Struktur	16
3	PDF-Erzeugung	18
3.1	Anforderungen	18
3.2	Erster Lösungsansatz	18
3.2.1	Text in PDF-Dokumenten	19
3.2.2	Fazit	21
3.3	Zweiter Lösungsansatz	22
3.3.1	Excel-Tabelle einlesen	22
3.3.2	Ersetzen der Platzhalter	22

3.3.3	Umwandlung in eine PDF-Datei	23
3.3.4	Klasse PDFGenerator	23
4	PDF-Signierung	24
4.1	Anforderungen	24
4.2	PDF-Signaturen, PAdES-Standard und eIDAS-Signaturen	25
4.3	PDF-Signaturen nach dem PAdES-Standard	25
5	Zusammenfassung und Ausblick	26
	Literaturverzeichnis	27
	Glossar	29
	Mathematische Symbole	30
	Selbstständigkeitserklärung	31

Abbildungsverzeichnis

2.1	Primfaktoren p und q sowie deren Produkt RSA-250	4
2.2	Prozess der Erzeugung und Validierung elektronischer Signaturen ..	8
2.3	Darstellung zur Public-Key Infrastruktur	11
4.1	Darstellung zur Public-Key Infrastruktur	25

Tabellenverzeichnis

2.1	Veraltete Hashfunktionen	6
2.2	Aktuell verwendete Hashfunktionen	6
3.1	Beispiel zu Glyphen in drei verschiedenen Fonts	20

Listings

2.1	Einträge in einem X.509 Zertifikat	12
2.2	Beispiel eines indirekten Objekts.....	13
2.3	Gültige Strings in einem PDF-Dokument	14
2.4	Gültige Namen in einem PDF-Dokument	14
2.5	Gültige Arrays in einem PDF-Dokument.....	14
2.6	Gültige Dictionarys in einem PDF-Dokument	14
2.7	Beispiel einer XRef Table	16
2.8	Beispiel eines PDF-Trailers	17
3.1	Definieren von drei verschiedenen Font Dictionaries in PDF-Dateien	20
3.2	Definieren von Text in PDF-Dateien	21
3.3	Beispiel einer document.xml	22

Einleitung und Problemstellung

Begonnen werden soll mit einer Einleitung zum Thema, also Hintergrund und Ziel erläutert werden.

Weiterhin wird das vorliegende Problem diskutiert: Was ist zu lösen, warum ist es wichtig, dass man dieses Problem löst und welche Lösungsansätze gibt es bereits. Der Bezug auf vorhandene oder eben bisher fehlende Lösungen begründet auch die Intention und Bedeutung dieser Arbeit. Dies können allgemeine Gesichtspunkte sein: Man liefert einen Beitrag für ein generelles Problem oder man hat eine spezielle Systemumgebung oder ein spezielles Produkt (z.B. in einem Unternehmen), woraus sich dieses noch zu lösende Problem ergibt.

Im weiteren Verlauf wird die Problemstellung konkret dargestellt: Was ist spezifisch zu lösen? Welche Randbedingungen sind gegeben und was ist die Zielsetzung? Letztere soll das beschreiben, was man mit dieser Arbeit (mindestens) erreichen möchte.

Grundlagen

2.1 Mathematische Grundlagen

Dieses Kapitel definiert grundlegende mathematische Begriffe und Probleme aus der Zahlentheorie. Zusammen betrachtet bilden sie das Fundament der Algorithmen zur Erstellung digitaler Signaturen. Die Kenntnis der arithmetischen Rechenoperationen, einschließlich Potenzieren und Logarithmieren wird vorausgesetzt.

2.1.1 Dualsystem (Binärsystem)

Definition 2.1. (*Dualsystem*)

Ganze Zahlen werden als Folge von Ziffern $z_m, z_{m-1}, \dots, z_0$ mit $m \in \mathbb{N}, z_i \in \{0, 1\}$ dargestellt. Der Wert W einer Binärzahl lässt sich wie folgt berechnen:

$$W = \sum_{i=0}^m z_i * 2^i \quad (2.1)$$

Beispiel 2.2. (Beispiel)

- $1011_2 = 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = 11_{10}$
- $110101_2 = 1 * 2^5 + 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 = 53_{10}$
- $10000000_2 = 1 * 2^7 + 0 * 2^6 + 0 * 2^5 + \dots + 0 * 2^0 = 128_{10}$

2.1.2 Hexadezimalsystem

Definition 2.3. (*Hexadezimalsystem*)

Ganze Zahlen werden als Folge von Ziffern $z_m, z_{m-1}, \dots, z_0$ mit $m \in \mathbb{N}_0, z_i \in \{0, 1, \dots, 9, a, b, \dots, f\}$ dargestellt. Den Buchstaben sind die Stellenwerte $a = 10, b = 11, c = 12, d = 13, e = 14$ und $f = 15$ zugeordnet. Der Wert W einer hexadezimalen Zahl lässt sich wie folgt berechnen:

$$W = \sum_{i=0}^m z_i * 16^i \quad (2.2)$$

Beispiel 2.4. (Beispiel)

- $24_{16} = 2 * 16^1 + 4 * 16^0 = 36_{10}$
- $c_{16} = 12 * 16^0 = 12_{10}$
- $ff_{16} = 15 * 16^1 + 15 * 16^0 = 255_{10}$

2.1.3 Division mit Rest

Bei der Division von ganzen Zahlen a und n bleibt ein Rest r . Dieser lässt sich mit der folgenden Formel berechnen.

Definition 2.5. (*Division mit Rest*)

Seien $a, n \in \mathbb{N}$ und $n > 0$, dann existieren zwei eindeutige Zahlen $q, r \in \mathbb{N}$ für die

$$a = n * q + r \quad (2.3)$$

gilt, mit $q = \lfloor \frac{a}{n} \rfloor$ und $0 \leq r < n$. Schreibweise: $r = a \bmod n$

Beispiel 2.6.

- $9 \bmod 6 = 3$
- $21 \bmod 5 = 1$
- $6 \bmod 2 = 0$

2.1.4 Wiederholtes Quadrieren

Das *Wiederholte Quadrieren* bietet eine Möglichkeit, effizient Zahlen der Form $M^e \bmod n$ zu berechnen. Der folgende Algorithmus stammt aus dem Original-Paper zu *RSA* [RSA78, S. 8].

Algorithmus 1: Wiederholtes Quadrieren

1. Es sei $e_k e_{k-1} \dots e_1 e_0$ die Binärdarstellung von e
 2. Setze die Variable C auf den Wert 1
 3. Wiederhole Schritte 3a und 3b für alle $i = k, k-1, \dots, 0$
 - 3a. Setze C auf den Wert von $C^2 \bmod n$
 - 3b. Wenn $e_i = 1$, dann setze C auf den Wert $C * M \bmod n$
 4. Wenn die vorherigen Schritte für alle $i \in \{0, \dots, k\}$ ausgeführt wurden entspricht $C = M^e \bmod n$.
-

2.1.5 Primzahlen und Teilbarkeit

Definition 2.7. (*Teilbarkeit*)

Seien $a, b \in \mathbb{Z}$, dann gilt

$$a|b \iff b = a * n \quad (2.4)$$

für ein $n \in \mathbb{Z}$.

Definition 2.8. (*Primzahlen*)

Eine Zahl $p \in \mathbb{N}$ ist eine Primzahl wenn gilt

1. $p > 1$
2. $\{a : a \in \mathbb{N} \wedge a|p\} = \{1, p\}$

Wörtlich ausgedrückt hat eine Primzahl p nur sich selbst und 1 als Teiler.

2.1.6 Größter gemeinsamer Teiler**Definition 2.9.** (*Größter gemeinsamer Teiler*)

Es seien $a, b \in \mathbb{Z}$, dann ist der größte gemeinsame Teiler (ggT) von a und b definiert durch

$$\text{ggT}(a, b) = \max\{x : x \in \mathbb{Z} \wedge x|a \wedge x|b\} \quad (2.5)$$

2.1.7 Probleme aus der Zahlentheorie**Faktorisierung**

Bei der *Faktorisierung* wird eine Zahl x in ihre Primfaktoren zerlegt. Dabei ist das Berechnen von x durch Multiplikation seiner Primfaktoren trivial – die Zerlegung hingegen nicht. Das folgende Beispiel veranschaulicht die Asymmetrie des Berechnungsaufwands deutlich.

Beispiel 2.10. Ein Computer berechnet das Ergebnis der Multiplikation von zwei 416-Bit Primzahlen im Bruchteil einer Sekunde, demgegenüber werden ca. 2700 physical core-years¹ benötigt um das 862-Bit Produkt wieder in seine Faktoren zu zerlegen[BGG⁺20].

Die verwendeten Zahlen (siehe Abbildung 2.1) waren $\text{RSA-250} = p * q$.

$\text{RSA-250} = 214032465024074496126442307283933356300861471514475501779775492$ $088141802344714013664334551909580467961099285187247091458768739$ $626192155736304745477052080511905649310668769159001975940569345$ $7452230589325976697471681738069364894699871578494975937497937,$ $p = 641352894770715802787901901705773890848250147429434472081168596$ $32024532344630238623598752668347708737661925585694639798853367,$ $q = 333720275949781565562260106053551142279407603447675546667845209$ $87023841729210037080257448673296881877565718986258036932062711$

Abbildung 2.1: Primfaktoren p und q sowie deren Produkt RSA-250

Quelle: Boudot et al., 2020 [BGG⁺20, S. 30]

¹ 1 physical core-year entspricht der ununterbrochenen Nutzung eines Prozessorkerns für ein Jahr

Diskreter Logarithmus

Es sei die Gleichung $a = g^e \bmod p$ gegeben und es sei p eine Primzahl. Daraus lassen sich folgende Szenarios konstruieren:

1. Szenario

Gegeben: $g, e, p \in \mathbb{N}$

Gesucht: a

2. Szenario

Gegeben: $g, a, p \in \mathbb{N}$

Gesucht: e

Die Berechnung der modularen Potenzierung im ersten Szenario ist mit Hilfe des *Wiederholten Quadrieren* trivial – das Finden des *diskreten Logarithmus* einer Zahl im zweiten Szenario ist es nicht.

Beispiel 2.11. $g = 2, e = 4, p = 13$

Berechne $a = 2^4 \bmod 13 = 16 \bmod 13 = 3$

Beispiel 2.12. $a = 6, g = 2, p = 13$

Finde ein e , so dass gilt $6 = 2^e \bmod 13$

2.2 Grundwerte der Informationssicherheit

Das Bundesamt für Sicherheit definiert die Grundwerte im Glossar des IT-Grundschutz-Kompendiums [Bun22] wie folgt:

Definition 2.13. (*Integrität*) Die Integrität schützt Daten vor der unautorisierten Modifikation Dritter und sorgt dafür, dass Veränderungen erkannt werden können. Dazu zählen das Verändern der Daten selber, Verfälschung der Angaben zum Autor und Manipulieren von Zeitangaben der Erstellung.

Definition 2.14. (*Authentizität*) Die Authentizität stellt sicher, dass der Absender beziehungsweise Autor von Daten tatsächlich derjenige ist, der er vorgibt zu sein.

Definition 2.15. (*Nichtabstreitbarkeit*) Die Nichtabstreitbarkeit stellt sicher, dass zwischen zwei Kommunikationspartnern weder die Herkunft einer Nachricht noch die Bestätigung des Erhalts im Nachhinein abgestritten werden kann.

Es existieren weitere zu schützende Werte wie Vertraulichkeit und Verfügbarkeit, jedoch sind diese nicht von Relevanz für diese Arbeit.

2.3 Hashfunktionen

Hashfunktionen dienen der sicheren Komprimierung von Daten und können durch die Berechnung eines „elektronischen Fingerabdrucks“ als Integritätsschutz verwendet werden.

Definition 2.16. (*Hashfunktionen*) Für eine Hashfunktion $H : X \rightarrow Y$ mit $X \subseteq \{0, 1\}^*$ und $Y = \{0, 1\}^n$ mit $n \in \mathbb{N}$ gilt:

1. $H(x)$ ist unkompliziert berechenbar für $x \in X$,
2. für ein $y \in Y$ ist es rechnerisch nicht möglich² ein $x \in X$ zu finden, dass $H(x) = y$ gilt,
3. für ein gegebenes $x_1 \in X$ ist es rechnerisch nicht möglich, ein $x_2 \in X$ zu finden, dass $H(x_1) = H(x_2)$ und $x_1 \neq x_2$ gilt, und
4. es ist rechnerisch nicht möglich, ein Paar $x_1, x_2 \in X$ zu finden, für das $H(x_1) = H(x_2)$ und $x_1 \neq x_2$ gilt.

Bedingungen 1 und 2 definieren eine Einwegfunktion, 3 die schwache und 4 die starke Kollisionsresistenz. Kollisionen treten auf, wenn bei zwei unterschiedlichen Eingaben derselbe Hashwert berechnet wird. Eine weitere Eigenschaft, die kryptografisch gute Hashfunktionen auszeichnet, ist, dass der Algorithmus bei minimaler Änderung der Eingabe eine völlig andere Ausgabe erzeugt (Lawineneffekt).

Algorithmus	Länge der Nachricht (Bits)	Länge des Hashwerts (Bits)
MD2, MD4, MD5	—	128
SHA-1	$< 2^{64}$	160

Tabelle 2.1: Veraltete Hashfunktionen

Algorithmus	Länge der Nachricht (Bits)	Länge des Hashwerts (Bits)
SHA-224, SHA-256	$< 2^{64}$	224, 256
SHA-384, SHA-512	$< 2^{128}$	384, 512
SHA3-224, SHA3-256	—	224, 256
SHA3-384, SHA3-512	—	384, 512
SHAKE128, SHAKE256	—	variabel

Tabelle 2.2: Aktuell verwendete Hashfunktionen

² *Rechnerisch nicht möglich* bedeutet in diesem Kontext, dass bei vertretbarem Zeitaufwand nicht alle Hashwerte berechnet werden können, bis eine Kollision gefunden wird.

2.4 Digitale Signaturen

Das Nationale Institute of Standards and Technology (NIST) definiert *Digitale Signaturen* als Ergebnis einer kryptografischen Transformation von Daten, sodass bei korrekter Implementierung die Einhaltung der Integrität, Authentizität und Nichtabstreitbarkeit nachvollziehbar verifiziert werden kann [Nat13].

Dies ist insbesondere in sensiblen Bereichen wie der Durchführung elektronischer Transaktionen oder dem Abschließen von Verträgen von großer Bedeutung, da es eine zuverlässige Möglichkeit bietet, die Herkunft und Unversehrtheit von Daten zu gewährleisten.

Aufgrund dieser Eigenschaft beschloss das Europäische Parlament in der Verordnung (EU) Nr. 910/2014 (eIDAS-Verordnung) unter anderem, dass „eine elektronische Signatur die gleiche Rechtswirkung wie eine handschriftliche Unterschrift haben sollte“ [Eur14]. Diese muss jedoch die Anforderungen einer *qualifizierten elektronischen Signatur* erfüllen um die volle Rechtswirkung zu entfalten.

2.4.1 Qualifizierte elektronische Signatur

Eine qualifizierte elektronische Signatur „[...] ist eine fortgeschrittene elektronische Signatur, die von einer qualifizierten elektronischen Signaturerstellungseinheit erstellt wurde und auf einem qualifizierten Zertifikat für elektronische Signaturen beruht.“ [Eur14].

Anforderungen an fortgeschrittene elektronische Signaturen

- Eindeutige Zuordnung des Unterzeichners
- Identifizierung des Unterzeichners möglich
- Signatur mit eindeutigen elektronischen Signaturerstellungsdaten erstellt
- Datenintegrität kann im Nachhinein nachvollziehbar überprüft werden

Anforderungen an qualifizierte elektronische Signaturerstellungseinheiten

- Vertraulichkeit der Signaturerstellungsdaten sind angemessen geschützt
- Sicherstellung der Sicherheit des verwendeten Verfahren und Signaturerstellungsdaten (z.B. Prüfung ob RSA-Schlüssellänge ≥ 2048)
- Signaturerstellungseinheit darf die zu unterzeichnenden Daten nicht verändern
- Erzeugung und Verwaltung von Signaturerstellungsdaten darf nur von einem qualifizierten Vertrauensdienstanbieter durchgeführt werden

Anforderungen an qualifizierte Zertifikate

Qualifizierte Zertifikate enthalten folgende Angaben:

- Angabe die den Aussteller des Zertifikats eindeutig repräsentiert
- Name oder Pseudonym des Unterzeichners
- Restliche Angaben die üblich für x509-Zertifikate sind (Gültigkeitsdauer, eindeutige Ordnungsnummer, Zertifikatsperlliste, usw.)

2.4.2 Überblick

Die folgende Abbildung 2.2 stammt aus dem *Digital Signature Standard* (DSS) und stellt vereinfacht den Ablauf der Erzeugung und Verifizierung elektronischer Signaturen dar. Im ersten Schritt wird in beiden Prozessen der *Message Digest* (Hashwert) der Nachricht ermittelt. Zusammen mit dem sogenannten *Private Key* (geheimer Schlüssel) berechnet die *Signature Generation*-Komponente (Signatur-Erstellung) aus dem Hashwert die Signatur. Diese dient gleichzeitig als Eingabe für die *Signature Verification*-Komponente (Signatur-Verifizierung). Bei gegebenem *Public Key* (öffentlicher Schlüssel) und Hashwert entscheidet die Verifikationsfunktion, ob die Signatur kryptografisch gültig ist.

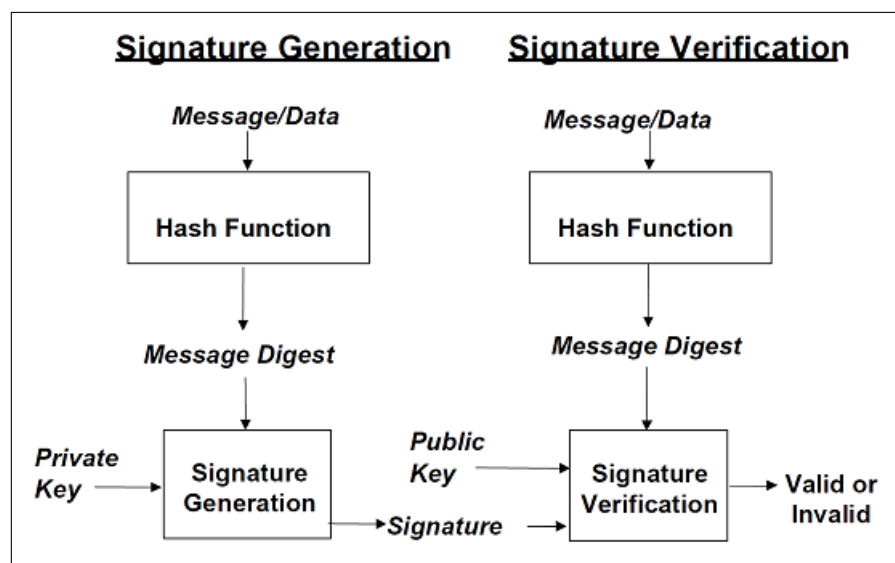


Abbildung 2.2: Prozess der Erzeugung und Validierung elektronischer Signaturen
Quelle: NIST, 2013 [Nat13, S. 9]

2.4.3 RSA (Rivest-Shamir-Adleman)

RSA ist ein Verfahren, benannt nach den Erfindern, um Daten zu verschlüsseln und digital zu signieren. Für diesen beiden Operationen wird ein Schlüsselpaar, bestehend aus *Public* und *Private Key*, benötigt. Die Sicherheit des Kryptosystems basiert zum Teil auf dem Problem der Faktorisierung (siehe 2.1.7) des öffentlichen Moduls n .

Schlüsselerzeugung

Der *öffentliche Schlüssel* besteht aus dem Zahlenpaar (e, n) und der *private Schlüssel* aus (d, n) . Um das Schlüsselpaar zu generieren müssen die folgenden Schritte, entnommen aus dem Original-Paper, ausgeführt werden [RSA78].

1. Wähle zwei zufällige Primzahlen p und q und berechne $n = p * q$.

2. Wähle zufällig eine natürliche Zahl d , für die gilt :

$$ggT(d, (p-1) * (q-1)) = 1 \quad (2.6)$$

3. Berechne die natürliche Zahl e mit p , q und d , für die gilt ³:

$$(e * d) \bmod ((p-1) * (q-1)) = 1 \quad (2.7)$$

Signieren einer Nachricht

Es sei M eine Nachricht und (d, n) ein privater Schlüssel. Die Signatur S zu M wird wie folgt berechnet:

$$S = M^d \bmod n \quad (2.8)$$

Verifizieren einer Signatur

Es sei M eine Nachricht, S die zugehörige Signatur und (e, n) ein öffentlicher Schlüssel.

$$x = S^e \bmod n \quad (2.9)$$

S ist eine gültige Signatur von M genau dann, wenn $x = M$ gilt.

³ e wird auch multiplikatives Inverse zu d bezeichnet

2.4.4 DSA (Digital Signature Algorithm)

DSA ist ein Verfahren zur digitalen Signierung und wurde 1991 im DSS veröffentlicht. Die Sicherheit des Algorithmus basiert auf dem diskreten Logarithmus (siehe 2.1.7) in endlichen Zahlkörpern. Ebenfalls wird hier ein Schlüsselpaar für das Signieren und Verifizieren benötigt. Die Erweiterung des Verfahrens auf elliptische Kurven wird auch als *Elliptic Curve Digital Signature Algorithm* (ECDSA) bezeichnet. Die Informationen zu diesem Algorithmus stammen aus dem Digital Signature Standard [Nat13].

Domain-Parameter

Für die Erzeugung des Schlüsselmaterials, Erstellung der Signatur und Verifikation werden die Parameter p , q und g benötigt. Dessen Berechnung hängt von der Wahl der Parameter L und N ab. Im DSS-Standard sind folgende Kombinationen gegeben:

$$L = 1024, N = 160$$

$$L = 2048, N = 224$$

$$L = 2048, N = 256$$

$$L = 3072, N = 256$$

Parameter p , q und g sind wie folgt definiert:

- p ist eine Primzahl mit der Bitlänge L , also $2^{L-1} < p < 2^L$.
- q ist eine Primzahl die $(p-1)$ teilt mit der Bitlänge N , also $2^{N-1} < p < 2^N$.
- g ist Generator einer Untergruppe der Ordnung q in der multiplikativen Gruppe $GF(p)$ mit $1 < g < p$.

Zusätzlich wird eine kryptografische Hashfunktion H benötigt.

Schlüsselerzeugung

Der *private Schlüssel* x ist eine zufällige ganze Zahl mit $0 < x < q$. Der *öffentliche Schlüssel* y berechnet sich aus x mit $y = g^x \mod p$.

Signieren einer Nachricht

Es sei M eine Nachricht, H eine kryptografische Hashfunktion, $outlen$ die Ausgabelänge von H in Bits und x ein privater Schlüssel. Für jede zu berechnende Signatur wird eine zufällige ganze Zahl k mit $1 < k < q$ gewählt. Die Signatur besteht aus dem Zahlenpaar (r, s) , welches wie folgt berechnet wird:

1. $r = (g^k \mod p) \mod q$
2. $z = \text{die linken } \min(N, outlen) \text{ Bits von } H(M)$
3. $s = (k^{-1} * (z + xr)) \mod q$

Sollte der Fall $r = 0$ oder $s = 0$ eintreten muss ein neues zufälliges k gewählt und Schritte 1. bis 3. erneut ausgeführt werden.

Anmerkung: Mit k^{-1} ist nicht der Bruch $\frac{1}{k}$ gemeint, sondern das multiplikative Inverse von k (also $k^{-1} * k \mod q = 1$).

Verifizieren einer Signatur

Es sei M eine Nachricht, (r, s) die zugehörige Signatur, H die verwendete Hashfunktion und (N, L, g, p, q) die verwendeten Parameter. Folgende Schritte werden ausgeführt, um die Gültigkeit der Signatur zu prüfen:

1. Prüfe, ob $0 < r < q$ und $0 < s < q$ gilt – wenn eine der beiden Bedingungen nicht zutrifft, ist die Signatur ungültig.
2. Wenn die Bedingungen aus Schritt 1 erfüllt sind, berechne die folgenden Werte:
 - $w = s^{-1} \bmod q$
 - $z = \text{die linken } \min(N, \text{outlen}) \text{ Bits von } H(M)$
 - $u1 = (z * w) \bmod q$
 - $u2 = (r * w) \bmod q$
 - $v = ((g^{u1} * y^{u2}) \bmod p) \bmod q$
3. Wenn $v = r$ zutrifft ist die Signatur gültig, ansonsten ist diese ungültig.

2.5 PKI (Public-Key Infrastructure)

Eine Public-Key Infrastructure (PKI) ist ein System, das der Erstellung und Verteilung von kryptografischem Schlüsselmaterial dient. Eine PKI-Infrastruktur umfasst verschiedene Komponenten wie Zertifizierungsstellen (CA), die digitale Zertifikate ausstellen und verwalten, Registrierungsstellen (RA), die die Identität von Zertifikatinhabern überprüfen, und Zertifikatverwaltungssysteme, die zur Verwaltung und Widerrufung (Revokation) von digitalen Zertifikaten verwendet werden [CSF⁺08]. Abbildung 2.3 zeigt, wie die teilnehmen Parteien in Verbindung stehen.

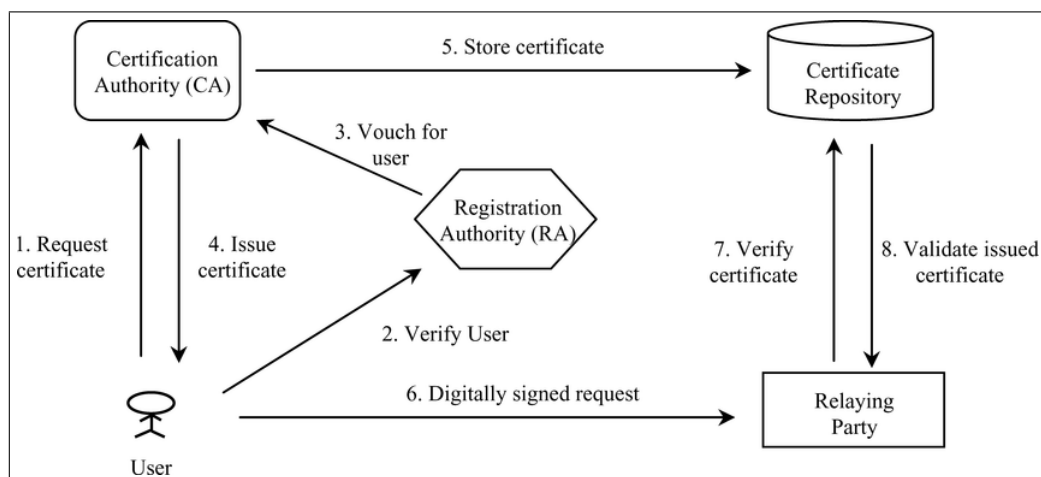


Abbildung 2.3: Darstellung zur Public-Key Infrastruktur

Quelle: Jie et al., 2011 [JAS⁺11, S. 12:3]

2.5.1 X.509 Version 3 Zertifikat

Ein X.509 Zertifikat ist in den meisten Fällen einer Person oder Entität zugeordnet. Diese Eindeutigkeit wird authentisch durch Dritte sichergestellt. Ein Zertifikat besteht aus den Werten:

1	Certificate ::= SEQUENCE	{
2	tbsCertificate	TBSCertificate,
3	signatureAlgorithm	AlgorithmIdentifier,
4	signatureValue	BIT STRING }

Listing 2.1: Einträge in einem X.509 Zertifikat

In dem Attribut *tbsCertificate* steht ein Wert vom Typ *TBSCertificate*. Darin enthalten sind unter anderem der öffentliche Schlüssel, Informationen über Gültigkeitsdauer, Aussteller und Besitzer. An Stelle von *signatureAlgorithm* wird der Algorithmus eingetragen, der verwendet wurde, um die Signatur des Zertifikats zu berechnen. Die dabei berechnete Signatur wird bei *signatureValue* eingetragen. Durch das Signieren des Zertifikats von der ausstellenden CA wird erreicht, dass der öffentliche Schlüssel genau einer Person oder Entität zugeordnet wird. Zusätzlich kann kein Angreifer ein Zertifikat mit der gleichen Identität und eigenem öffentlichen Schlüssel erstellen, da die Signatur mit dem privaten Schlüssel der CA berechnet wird und dieser geheim gehalten wird [CSF⁺08].

2.5.2 Gültigkeitsmodelle

Gültigkeitsmodelle dienen dazu, die Gültigkeit der signierenden Zertifikate zusätzlich bei der Verifikation von Signaturen einzubeziehen. Dabei wird in drei etablierte Modelle unterschieden: Schalenmodell, Modifiziertes Schalenmodell (Hybridmodell) und Kettenmodell. Das mittlerweile außerkraftgetretene Signaturgesetz (SigG) verlangte die Verwendung des Kettenmodells zur Valierung elektronischer Signaturen. Die eIDAS-Verordnung stellt keine direkten Anforderungen an die Verwendung eines konkreten Gültigkeitsmodells, jedoch schließen die in Artikel 32 („Anforderungen an die Validierung qualifizierter elektronischer Signaturen“) beschriebenen Voraussetzungen die Verwendung des Schalen -und Kettenmodells aus [Eur14]. Daraus kann gefolgert werden, dass das modifizierte Schalenmodell geeignet ist, elektronische Signaturen eIDAS-konform zu validieren.

Schalenmodell

Die Validierung mit dem Schalenmodell ist erfolgreich, wenn die Signatur und das signierende Zertifikat (inkl. aller Zertifikate im Zertifikatspfad) zum Prüfungszeitpunkt gültig ist.

Modifiziertes Schalenmodell

Konträr zum Schalenmodell wird die Gültigkeit der Zertifikate nicht zum Prüfungszeitpunkt, sondern zum Signierzeitpunkt betrachtet.

Kettenmodell

Die Validierung mit dem Schalenmodell ist erfolgreich, wenn die Signatur gültig ist und das signierende Zertifikat (inkl. aller Zertifikate im Zertifikatspfad) innerhalb der Gültigkeitsdauer des im Zertifikatspfad voranstehenden Zertifikat ausgestellt wurde.

2.6 PDF (Portable Document Format)

Eine PDF-Datei entspricht der digitalen Form eines Dokumentes und wird im Standard ISO 32000 spezifiziert. Initial wurde dieses Datei-Format von dem Softwareunternehmen Adobe ⁴ entwickelt. Seit 2008 wird der Standard durch die Organisation *International Organization for Standardization* (ISO) weiterentwickelt. Die neueste Version wurde 2020 unter dem Namen ISO 32000-2:2020 veröffentlicht und definiert die PDF-Version 2.0. Alle nachfolgenden Informationen zu PDF-Dokumenten entstammen aus dem zugehörigen Standard (ISO 32000-1:2008) [ISO08].

2.6.1 Objekte

Im PDF-Standard werden acht Basistypen von Objekten definiert: boolesche Werte, numerische Objekte (ganze und reelle Zahlen), Strings, Names, Arrays, Dictionaries, Streams und ein Null-Objekt. Objekte können entweder *direkt* oder *indirekt* definiert werden. Indirekte Objekte werden durch eine Objekt- und Generationsnummer referenziert. Zusätzlich umgeben die beiden Wörter *obj* und *endobj* den Wert des Objektes.

```
1  5 0 obj
2      (Objekt)
3  endobj
4  ...
6  5 0 R
```

Listing 2.2: Beispiel eines indirekten Objekts

Im oberen Beispiel wird ein indirektes String-Objekt mit dem Wert „Objekt“ definiert. Die Objekt- und Generationsnummer sind die Zahlen 5 und 0. In Zeile 6 wird das Objekt durch diese beiden Werte mit dem Buchstaben *R* referenziert. Anstelle von einer Referenz auf das indirekte Objekt kann auch dessen Wert dort stehen. In dem Fall wäre es ein direktes Objekt.

Strings

String-Objekte enthalten Zeichenketten und sind mit `()` und `<>` umgeben. Die runden Klammern kennzeichnen Strings, die eine beliebige Menge von Schriftzeichen

⁴ www.adobe.com/

enthalten. Die Vergleichsoperatoren geben an, dass es sich um einen hexadezimalen String handelt.

```
1  (Dies ist ein String)
2  ()
3  <911FA0>
4  <901FA>
```

Listing 2.3: Gültige Strings in einem PDF-Dokument

Namen

Namen definieren einzigartige Symbole und werden durch eine Folge von beliebigen Schriftzeichen, beginnend mit einem Slash, definiert.

```
1  /Name1
2  /.notdef
3  /A#42
```

Listing 2.4: Gültige Namen in einem PDF-Dokument

Arrays

Ein Array umfasst eine Liste von sequenziell angeordneten Objekten, die von zwei eckigen Klammern umgeben sind.

```
1  [549 3.14 false (Some String) /SomeName]
2  []
3  [[(A)][(nested)][(Array)]]
```

Listing 2.5: Gültige Arrays in einem PDF-Dokument

Dictionaries

Ein Dictionary-Objekt enthält mehrere Objekt-Paare, die als Einträge bezeichnet werden. Das erste Element eines Eintrags wird als *Key* und das zweite als *Value* bezeichnet. Der Key ist auf den Objekt-Typ *Name* beschränkt, wohingegen der des Value beliebig ist. Außerdem sind Dictionaries mit den Zeichen <<>> umgeben.

```
1  << /Type /Beispiel
2    /Subtype /DictionaryBeispiel
3    /Version 1.0
4    /Value (Wert)
5    /SubValue << /Item1 0.4
6                /Item2 true
7                >>
8  >>
```

Listing 2.6: Gültige Dictionaries in einem PDF-Dokument

Streams

Objekt-Streams dienen dazu mehrere indirekte Objekt in einem Stream-Objekt zu speichern. Zusätzlich kann ein Filter auf diesen angewendet werden, um die Größe des Streams zu komprimieren.

2.6.2 Datei-Struktur

Eine normgerechte PDF-Datei besteht aus einem *Header*, *Body*, *XRef table* (Cross-Reference) und einem *Trailer*.

Header

Der *Header* steht in der ersten Zeile einer PDF-Datei und spezifiziert die zugrunde liegende PDF-Version. Der Wert beginnt mit den fünf Zeichen %PDF-, gefolgt von der Version in Form von 1.*N* mit $N = 0, 1, \dots, 7$ (mittlerweile auch 2.0).

Body

Der *Body* spezifiziert mit indirekten Objekten den Inhalt eines Dokuments.

Cross-Reference Table

In der *Cross-Reference Table* werden die Positionen der Objekte innerhalb einer PDF-Datei gespeichert. Dadurch kann auf den Wert eines Objektes zugegriffen werden, ohne die ganze Datei einzulesen. Für jedes indirekte Objekt existiert ein einzeliger Eintrag, der den Byte-Offset dieses Objekts innerhalb der Datei angibt. Jeder Eintrag hat folgendes Format:

nnnnnnnnnn gggg n/f eol

mit:

nnnnnnnnnn ist ein Byte-Offset (Dezimalzahl mit 10 Ziffern)

ggggg ist die Generationsnummer (Dezimalzahl mit 5 Ziffern)

n wenn der Eintrag belegt ist, *f* wenn der Eintrag frei ist

eol ist ein Schriftzeichen, welches das Ende der Zeile signalisiert

```

1  xref
2  0 6
3  0000000003 65535 f
4  0000000017 00000 n
5  0000000081 00000 n
6  0000000000 00007 f
7  0000000331 00000 n
8  0000000409 00000 n

```

Listing 2.7: Beispiel einer XRef Table

Die beiden Zahlen in Zeile zwei spezifizieren, dass 6 Einträge angegeben werden, die bei Objekt-Nummer 0 beginnen. Aus den folgenden Einträgen lässt sich also ablesen, dass das Objekt mit der Nummer 0 nicht belegt ist und an Byte-Position 3 in der Datei liegt. Das Objekt mit der Nummer 1 ist belegt und an Byte 17 in der Datei positioniert.

Die Cross-Reference Table kann auch als Stream gespeichert werden, sodass ein Cross-Reference Stream entsteht.

Trailer

Der *Trailer* einer PDF-Datei signalisiert das Ende des Dokuments, speichert die Position der Cross-Reference Table und referenziert den *Document Catalog*.

```
1  trailer                                % Schluesselwort - signalisiert Beginn des Trailers
2  << /Size 22                          % Anzahl der Eintraege in der Cross-Reference Table
3    /Root 2 0 R                        % Indirekte Objekt-Referenz auf den Document Catalog
4  >>
5  startxref                            % Schluesselwort
6  18799                                % Byte-Position der Cross-Reference Table
7  %%EOF                                % Schluesselwort - signalisiert Ende des Trailers
```

Listing 2.8: Beispiel eines PDF-Trailers

PDF-Erzeugung

Im Rahmen dieser Arbeit soll eine Applikation erstellt und konzeptioniert werden, die unter anderem eine Word-Vorlage mit gegebenen Werten füllt und anschließend als eine PDF-Datei abspeichert. Dabei wird in den folgenden Abschnitten auf die Anforderungen eingegangen und zwei mögliche Lösungsansätze präsentiert. Alle Informationen in diesem Kapitel, die PDF-Dateien betreffen, wurden aus der ISO-Norm *ISO 32000-1:2008* entnommen [ISO08]. Alle Informationen zu Word-Dateien wurden aus der ISO-Norm *ISO/IEC 29500-1:2016* entnommen [ISO16].

3.1 Anforderungen

Es ist eine Word-Vorlage gegeben, die an gewissen Stellen Platzhalter enthält, welche durch Sonderzeichen gekennzeichnet sind. In diesem Fall handelt es sich um französische Anführungszeichen (« und »). Anstelle dessen könnten ebenfalls beliebig andere Sonderzeichen dienen, welche nicht regulär in der Vorlage erscheinen.

In einer Excel-Tabelle stehen alle Werte für die jeweiligen Platzhalter. Dabei ist eine Zeile jeweils einem Dokument zugeordnet und enthält alle benötigten Werte. In der Vorlage gibt es zum Beispiel einen Abschnitt der lautet: „geboren am «Geburtsdatum» in «Geburtsort» “. Eine entsprechende Excel-Tabelle enthält dann unter anderem zwei Spalten mit den Namen *Geburtsdatum* und *Geburtsort*.

3.2 Erster Lösungsansatz

Eine Möglichkeit das Problem, unter Erfüllung der gegebenen Anforderungen, zu lösen ist, die Word-Vorlage erst in ein PDF-Dokument umzuwandeln und anschließend die Platzhalter im Text zu ersetzen. Das erweist sich im Vergleich zum zweiten Ansatz als sehr aufwendig. In der finalen Implementierung wird deswegen Zweites angewendet, sodass dieses Konzept nur der Vollständigkeit halber aufgeführt wird.

3.2.1 Text in PDF-Dokumenten

In PDF-Dokumenten wird jeder Buchstabe eines Texts durch sogenannte *Glyphen* repräsentiert. Eine Glyphe ist im Grunde eine grafische Form, die zu einer Font gehört und ein Zeichen repräsentiert. Fonts enthalten Zuordnungen von Schriftzeichen eines bekannten Zeichensatzes (Unicode, ASCII) zu den Glyphen.

	Courier	Helvetica	Times
ASCII 41 ₁₆ : A	A	A	A
Unicode DC ₁₆ : Ü	Ü	Ü	Ü

Tabelle 3.1: Beispiel zu Glyphen in drei verschiedenen Fonts

Zum Beispiel sind die Glyphen A, **A** und **A** in Tabelle 3.1 die grafischen Ausprägungen des Schriftzeichens „A“. Zusätzlich zur Form und Aussehen Symbols werden weitere Maße wie Breite, Höhe und Position des nächsten Glyphen gespeichert. Soll das Wort „Informatik“ als Text dargestellt werden, muss bekannt sein wo das „n“ nach dem „I“ positioniert wird. Die Informationen, wie ein Buchstabe grafisch gezeichnet wird, ist in einem *Font Program* enthalten. Dieses ist entweder in einem PDF-Dokument eingebettet, in einem PDF-Reader oder einer externen Font-Datei enthalten. Das *Font Dictionary* verwaltet Einträge, die den Typ der Font, dessen Encodierung und den Namen des Font Programs spezifizieren.

Beispiel 3.1. („Informatik“ in den Fonts Courier, Helvetica und Times-Roman)

Informatik
Informatik
Informatik

```

1      10 0 obj
2      << /Font <<
3          /F1 11 0 R
4          /F2 12 0 R
5          /F3 13 0 R
6      >>
7      >>
8      endobj

10     11 0 obj
11     <<
12         /Type /Font
13         /Subtype /Type1
14         /BaseFont /Courier
15     >>
16     endobj

18     12 0 obj
19     <<
20         /Type /Font
21         /Subtype /Type1
22         /BaseFont /Helvetica
23     >>
24     endobj

26     13 0 obj
27     <<
28         /Type /Font
29         /Subtype /Type1
30         /BaseFont /Times-Roman
31     >>
32     endobj

```

Listing 3.1: Definieren von drei verschiedenen Font Dictionaries in PDF-Dateien

Beliebiger Text kann nun in den drei definierten Schriftarten dargestellt werden. Dieser wird in ein Stream-Objekt geschrieben und durch die beiden Schlüsselwörter „BT“ (*Begin Text*) und „ET“ (*End Text*) eingeschlossen.

```

1      14 obj
2      <</Length 205>>                                % Laenge des Streams in Bytes
3      stream
4      BT
5          /F1 17 Tf                                     % Font-Name Font-Groesse Tf
6          1 0 0 1 10 700 Tm                             % ... x-Koord. y-Koord. Tm
7          (Informatik) Tj                               % String-Objekt Tj

9          /F2 17 Tf
10         0 -20 Td                                       % x-Koord. y-Koord. Td
11         [(Inf)30(or)-25(matik)] TJ                   % Array von String-Objekten

13         /F3 17 Tf
14         0 -20 Td
15         [<0049> <006E> <0066> <006F> <0072> ... <0074> <0069> <006B>] TJ

16     ET
17     endstream
18     endobj

```

Listing 3.2: Definieren von Text in PDF-Dateien

Der Text wird mit sogenannten *Operanden* und *Operatoren* definiert. Mit dem Operator „Tf“ wird eine Font referenziert und dessen Schriftgröße spezifiziert. In Zeile sechs des Listings 19 wird mit dem Operator „Tm“ der *Text-State* festgelegt. Dieser verwaltet eine Matrix mit der mathematische Transformationen wie Skalierung, Rotation und Verschiebung auf den Text angewendet werden. In den Zeilen 7, 11 und 15 befinden sich drei verschiedene Varianten, wie Text dargestellt werden kann. Der Operator „Tj“ nimmt genau ein String-Objekt als Operanden entgegen. Mit „TJ“ wird Text als Array von String-Objekten dargestellt, wobei die horizontale Position für alle Teilwörter individuell angepasst werden kann (siehe Listing 19 Zeile 11, „or“ wird um 30 Einheiten nach rechts verschoben). Einzelne Schriftzeichen können auch direkt als Glyphen über ihren Index im Font Program angegeben werden (siehe Listing 19 Zeile 15). Die Information welcher Buchstabe letztendlich durch die Glyphen dargestellt wird, ist in dem Font Dictionary enthalten, welches vorher mit dem Tf-Operator referenziert wurde.

3.2.2 Fazit

Dieses kleine Beispiel zeigt bereits den Umfang der Textdarstellung in PDF-Dokumenten. Es existiert aktuell keine Python-Bibliothek, die eine Schnittstelle für das Ändern von Text auf einer niedrigen Abstraktionsebene anbietet. Das eigene Implementieren dieser Funktionalitäten würde den Umfang einer Bachelorarbeit sprengen und auch nicht wirklich der Problemstellung entsprechen. Die einzelnen Platzhalter müssen also ersetzt werden bevor die Vorlage in ein PDF-Dokument umgewandelt wird.

3.3 Zweiter Lösungsansatz

Für das *Office Open XML Document*-Format (Dateiendung .docx) existiert seit November 2008 ebenfalls eine ISO-Norm. Dieser Standard wird von Microsoft und der Ecma International entwickelt und unter dem Namen ISO/IEC 29500 veröffentlicht. Die letzte Version erschien 2016, wobei der Entwicklungsprozess eines neuen Entwurfs bereits begonnen hat.

Word-Dokumente bestehen aus ZIP-komprimierten XML-Dateien. Die *Extensible Markup Language* (XML) stellt Daten hierarchisch mit sogenannten *Elementen* und *Tags* dar. Jedes Word-Dokument enthält die Datei *document.xml*, welche den textuellen Inhalt ausmacht. Der eigentliche Text steht innerhalb von *t*-Elementen (Text). Diese werden wiederum von *r*-Elementen (Runs) umgeben. Hinsichtlich des Textes befinden sich Paragraphen auf der obersten Hierarchieebene und umschließen die *Runs*.

```
1  <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2  <w:document>
3      <w:body>
4          <w:p>
5              <w:r>
6                  <w:t xml:space="preserve">Beispiel-Text</w:t>
7              </w:r>
8          </w:p>
9      </w:body>
10 </w:document>
```

Listing 3.3: Beispiel einer document.xml

Ausführlicher muss das *Office Open XML Document*-Format hier nicht weiter erläutert werden, da eine Python-Bibliothek (*python-docx*) existiert, die den zugehörigen ISO-Standard implementiert. Das weitere Vorgehen wird in drei Schritten eingeteilt: Excel-Tabelle einlesen, Platzhalter im Word-Dokument mit den tatsächlichen Werten aus der Excel-Datei ersetzen, Word-Dokument in eine PDF-Datei umwandeln.

3.3.1 Excel-Tabelle einlesen

Der erste Schritt ist dank der Python-Bibliothek *openpyxl* relativ unkompliziert. Diese ermöglicht das zeilenweise Iterieren der Excel-Tabelle, sodass die Platzhalter-Werte-Paare in eine Liste von Dictionaries gespeichert werden können. Die Funktion *parse_excel_file* im zugehörigen Code dieser Ausarbeitung ist eine Implementierung zu diesem Verfahren. Zurückgegeben wird eine Liste von Dictionaries mit Einträgen der Form («Platzhalter»:Wert).

3.3.2 Ersetzen der Platzhalter

Zuerst wird das Word-Dokument mit *python-docx* in ein Document-Objekt geparkt. Darin enthalten sind zwei Attribute, die jeweils Listen zu allen Paragraphen und Tabellen innerhalb der Word-Datei verwalten. Es reicht nicht, an diesem Punkt

nur durch alle Paragraphen zu iterieren, da diese auch eingebettet in einer Tabellenzelle vorkommen können. Als Nächstes wird die im vorherigen Abschnitt beschriebene Funktion aufgerufen. Jeder Listeneintrag in dessen Rückgabewert entspricht einer neuen Word-Datei. Der folgende Ablauf wird somit für jedes Listenelement ausgeführt.

Das ganze Dokument wird bis zu der Paragraphen-Ebene durchlaufen. Anschließend wird die Funktion *replace_text_in_paragraph* aufgerufen, um alle Platzhalter innerhalb eines Paragraphen durch ihre Werte zu ersetzen. Dabei kann es vorkommen, dass ein Platzhalter angegeben ist, jedoch kein Wert für diesen in der Excel-Tabelle definiert wird. In diesem Fall wird der Stellvertreter mit einem leeren String ersetzt.

Wenn dieser Schritt vollendet ist, wird eine Kopie der Word-Datei mit den ersetzten Werten unter einem angegebenen Pfad abgespeichert.

3.3.3 Umwandlung in eine PDF-Datei

Word bietet eine Funktionalität, um deren Dokumente in PDF-Dateien umzuwandeln. Dafür müsste jedes Dokument einzeln geöffnet und händisch umgewandelt werden. Dieser Schritt lässt sich jedoch mit einem *Visual Basic Script* (VBS) automatisieren. Im Code zu dieser Ausarbeitung befindet sich die Datei *DocxToPdf.vbs*, welche genau dieses Problem löst. Das Skript bekommt einen Ordnerpfad und einen booleschen Wert als Input. Zunächst wird, falls nicht vorhanden, eine Programminstanz von Word gestartet. Anschließend wird jede Datei mit Endung „.docx“ in der Programminstanz geöffnet, von Word umgewandelt und im gleichen Ordner abgespeichert. Ist der boolesche Wert mit wahr belegt, wird die Word-Datei nach der Umwandlung gelöscht werden. Wenn dieser nicht mit wahr belegt ist, wird das Dokument nicht gelöscht und es können noch Fehler, die vielleicht bei der Umwandlung auftreten, behoben werden.

3.3.4 Klasse PDFGenerator

Die Klasse *PDFGenerator* der zugehörigen Datei *PDFGenerator.py* kombiniert alle zuvor genannten Schritte in einer Implementierung. Dabei wird der Aufruf der automatischen Word-Umwandlung in einem eigenen Thread gestartet, um das Blockieren der GUI zu verhindern.

PDF-Signierung

In diesem Abschnitt wird erläutert, wie PDF-Dokumente digital signiert werden. Es wird ebenfalls der PAdES-Standard vorgestellt, welcher eine Möglichkeit bietet, die Gültigkeitsdauer der Signaturen zu erweitern.

4.1 Anforderungen

Die zu implementierende Anwendung soll PDF-Dokumente digital signieren können. Durch die Einbettung der Signaturen in eine PDF-Datei wird die Unterstützung der ISO-Norm ISO 32000-1 implizit vorausgesetzt. Es soll ebenfalls die Unterstützung des PAdES-Standards (ETSI EN 319 143) gewährleistet werden. Die elektronisch signierten PDF-Dokumente sollen auf gängigen PDF-Readern mit Signatur-Verifikation erkannt und verifiziert werden können.

Es existieren aktuell keine Studien zu den Nutzerstatistiken der etablierten PDF-Reader. An diesem Punkt wird somit angenommen, dass *Adobe Acrobat Reader* die mit Abstand am weitesten verbreitete Anwendung ist. Ein Grund zu dieser Annahme ist, dass Microsoft ab März 2023 Adobe-Technologien in dem Browser *Microsoft Edge* integriert [Mic23].

Die eIDAS-Konformität ist zwar nicht gefordert, jedoch in Anbetracht der bereits erwähnten gesetzlichen Anerkennung ein erstrebenswertes Ziel.

Die im Folgenden genannten Anforderungen wurden als sekundär betrachtet. Das Hauptziel war es, zuerst eine Anwendung zu implementieren, die beliebig viele PDF-Dateien mit importiertem Schlüsselmaterial (vorerst nur RSA-Unterstützung) nach dem PAdES-Standard digital signiert. Alle weiteren Funktionen dienen nur der Verbesserung des Benutzererlebnisses.

Schlüsselmaterial soll generierbar sein und importiert werden können. Es sollen verschiedene asymmetrische Kryptoverfahren wie RSA und ECDSA unterstützt werden. Die Anwendung soll außerdem ein Logo und eine handschriftliche Unterschrift als Unterschriftenfeld in die PDF-Datei einfügen können.

Ist eine der primären Anforderungen nicht erfüllt, wird dies ausführlich erklärt. Wird eine der sekundären Anforderungen nicht implementiert, wird dies dem Zeitmangel zugeschrieben.

4.2 PDF-Signaturen, PAdES-Standard und eIDAS-Signaturen

Der Zusammenhang zwischen PDF-Signaturen, dem PAdES-Standard und eIDAS-Signaturen lässt sich am Besten mit dem folgenden Venn-Diagramm veranschaulichen. Die grau ausgefüllte Fläche stellt die Menge der gewünschten Signaturen dar.

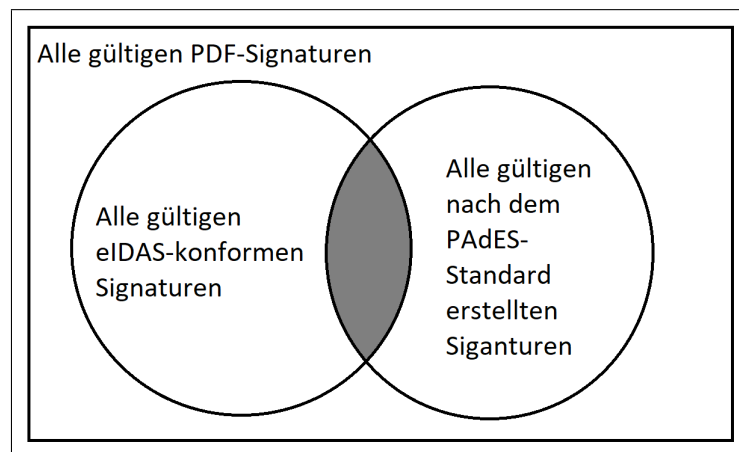


Abbildung 4.1: Darstellung zur Public-Key Infrastruktur
Quelle: Eigene Darstellung

Die beiden Mengen innerhalb des Rechtecks stellen beide gültige PDF-Signaturen dar. Es werden lediglich Werte, die Attribute innerhalb der Signaturen annehmen können, durch die beiden Spezifikationen beschränkt.

4.3 PDF-Signaturen nach dem PAdES-Standard

Die *European Telecommunications Standards Institute* (ETSI) ist wie ISO eine Organisation, die die Standardisierung in verschiedenen Bereichen vorantreibt. In diesem Fall ist ETSI besonders im Bereich der Telekommunikation tätig. Darunter fällt auch die Entwicklung des PAdES-Standards (ETSI EN 319 142) und allen zugehörigen Spezifikationen zu digitalen Signaturen in elektronischen Dokumenten.

Die eigentliche Signatur mit ihren Metadaten wird innerhalb der PDF-Datei auf zwei verschiedenen Ebenen verwaltet – auf der Ebene von PDF-Objekten und auf der des CMS-Objekts (Cryptographic Message Syntax).

Zusammenfassung und Ausblick

In diesem Kapitel soll die Arbeit noch einmal kurz zusammengefasst werden. Insbesondere sollen die wesentlichen Ergebnisse Ihrer Arbeit herausgehoben werden. Erfahrungen, die z.B. Benutzer mit der Mensch-Maschine-Schnittstelle gemacht haben oder Ergebnisse von Leistungsmessungen sollen an dieser Stelle präsentiert werden. Sie können in diesem Kapitel auch die Ergebnisse oder das Arbeitsumfeld Ihrer Arbeit kritisch bewerten. Wünschenswerte Erweiterungen sollen als Hinweise auf weiterführende Arbeiten erwähnt werden.

Literaturverzeichnis

- BGG⁺20. BOUDOT, FABRICE, PIERRICK GAUDRY, AURORE GUILLEVIC, NADIA HENINGER, EMMANUEL THOMÉ und PAUL ZIMMERMANN: *Comparing the Difficulty of Factorization and Discrete Logarithm: A 240-Digit Experiment*. In: *Advances in Cryptology – CRYPTO 2020*, Seiten 62–91. Springer International Publishing, 2020.
- Bun22. BUNDESAMT FÜR SICHERHEIT IN DER INFORMATIONSTECHNIK: *IT-Grundschutz-Kompendium*. Reguvis, 2022.
- CSF⁺08. COOPER, D., S. SANTESSON, S. FARRELL, S. BOEYEN, R. HOUSLEY und W. POLK: *RFC 5280. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. Technischer Bericht, Mai 2008.
- Eur14. EUROPÄISCHES PARLAMENT UND RAT: *Verordnung (EU) Nr. 910/2014*, Juli 2014.
- ISO08. *ISO 32000-1:2008. Document management — Portable document format — Part 1: PDF 1.7*. Standard, International Organization for Standardization, Juli 2008.
- ISO16. *ISO/IEC 29500-1:2016. Information technology — Document description and processing languages — Office Open XML File Formats — Part 1: Fundamentals and Markup Language Reference*. Standard, International Organization for Standardization, November 2016.
- JAS⁺11. JIE, WEI, JUNAID ARSHAD, RICHARD SINNOTT, PAUL TOWNEND und ZHOU LEI: *A review of grid authentication and authorization technologies and support for federated access control*. ACM Computing Surveys, 43(2):1–26, Januar 2011.
- Mic23. MICROSOFT EDGE TEAM: *Adobe and Microsoft Bring Industry-Leading Acrobat PDF Experience to 1.4 Billion Windows Users through Microsoft Edge*. <https://web.archive.org/web/20230217101250/https://blogs.windows.com/msedgedev/2023/02/08/adobe-acrobat-microsoft-edge-pdf/>, Februar 2023. Zugegriffen: 19.02.2023.
- Nat13. NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY: *Digital Signature Standard (DSS)*. Technischer Bericht, Juli 2013.

-
- RSA78. RIVEST, R. L., A. SHAMIR und L. ADLEMAN: *A method for obtaining digital signatures and public-key cryptosystems*. Communications of the ACM, 21(2):120–126, Februar 1978.

A

Glossar

Algorithmus	Abfolge von Rechenschritten zur Lösung eines gegebenen Problems
ASCII	Zeichensatz, Nummerierung ($0_{10} - 127_{10}$) von gängigen westeuropäischen Schriftzeichen
Bit	Zahl in Binärdarstellung mit dem Wert 0 oder 1
Bitfolge	Folge von Bits beliebiger Länge
Boolesche Werte	Wahrheitswert, entweder wahr oder falsch
Byte	Gruppierung von 8 Bits
CAdES	CMS Advanced Electronic Signatures
Font	Schriftart, z.B. Arial oder Verdana
Klartext	Nicht verschlüsselte/r Daten/Text
Kryptografie	Teilgebiet der Informatik – Erarbeitung und Beurteilung von Verfahren zur Sicherung geheimer Daten
PADES	PDF Advanced Electronic Signatures
Unicode	Zeichensatz, Nummerierung ($0000_{16} - FFFF_{16}$) des Großteils aller Schriftzeichen
ZIP-komprimiert	Verlustfreie Komprimierung von Daten – Name entstammt von dem englischen Begriff <i>zipper</i>

B

Mathematische Symbole

$\mathbb{N}_0$	Natürliche Zahlen (inklusive der 0)
$\mathbb{Z}$	Ganze Zahlen
$x \in \mathbb{Z}$	Beliebige Zahl x aus den ganzen Zahlen
$\sum_0^n x$	Summe der Zahlen 0 bis n, $0 + 1 + 2 + \dots + n$
$a b$	a teilt b
$\iff$	genau dann, wenn
$\{a : b\}$	Menge der Zahlen a , die Bedingung b erfüllen
$\wedge$	Logisches Und
$\lfloor x \rfloor$	Abrundungsfunktion, rundet zur nächstkleineren ganzen Zahl ab
$a \bmod n$	„a reduziert modulo n“, siehe Definition 2.5
$GF(p)$	Galois-Feld von p

C

Selbstständigkeitserklärung

- ☐ Diese Arbeit habe ich selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet.
- ☐ Diese Arbeit wurde als Gruppenarbeit angefertigt. Meinen Anteil habe ich selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet.

Namen der Mitverfasser:

Meine eigene Leistung ist:

Datum

Unterschrift der Kandidatin/des Kandidaten